

Backend Developer Guide

From an empty repository to a production-ready backend, module by module.

24

Modules, in build order

35

Database tables

95

REST APIs + realtime events

8

Delivery phases

Stack

NestJS • TypeScript • MySQL
8 • Redis • S3

Audience

Backend developers joining
the project

Version

1.0 • 30 September 2026

Contents

1. Read this first	3
2. The product in one page	3
3. Architecture	4
4. Engineering conventions	5
5. Security baseline	6
6. Scalability plan	7
7. Build order: all modules at a glance	7
8. Module specifications	9
Phase 0 • Foundation (M01–M03)	9
Phase 1 • Identity (M04–M07)	10
Phase 2 • Profile & live verification (M08–M12)	13
Phase 3 • Safety & moderation (M13–M15)	17
Phase 4 • Dating core (M16–M18)	20
Phase 5 • Communication (M19–M20)	22
Phase 6 • Monetization & trust+ (M21–M23)	24
Phase 8 • Post-launch (M24)	27
9. Appendices	28
A. Domain events	28
B. Background jobs	28
C. Error codes	28
D. Environment variables	29
E. Testing and Definition of Done	29

1 Read this first

This guide is the single source of truth for building the Kuchu Puchu backend. A new developer should be able to start at module M01 and finish at M24 without needing any other document. Read sections 1–7 once. After that, work through section 8 one module at a time, in order.

24

modules

35

tables

95

REST endpoints

~20

domain events

~17

background jobs

How each module section is organised

Every module in section 8 follows the same layout, so you always know where to look:

Part	What you find there
Header	Module code (M01...M24), phase, dependencies, number of tables and APIs
Purpose	What the module is responsible for, in one or two sentences
Entities	Every table with its columns, types, rules and indexes. Standard columns are not repeated (see 4.3).
APIs	Method, path, who may call it, and what it does. Request and response details follow the conventions in section 4.
Rules	Business rules, validations and limits the service must enforce
Done when	The checklist a reviewer uses before the module is merged

Words used in this guide

Term	Meaning
Verified user	A user whose live selfie is approved. Most dating features require this (enforced by <code>VerifiedUserGuard</code>).
Entitlement	A feature or quota unlocked by a plan, for example <code>see_who_liked_me</code> or <code>daily_likes = 100</code> . Code checks entitlements, never plan names.
Outbox event	A row written in the same database transaction as a change, later delivered to handlers (push, realtime, cleanup).
Silent 404	Blocked, hidden and deleted users all return the same 404 to others, so nobody can learn that they were blocked.
Access column	Public = no token • User = any signed-in user • Verified = selfie approved • Mod = moderator or admin • Admin = admin only • Hook = signed webhook from a provider

Configuration first. Every number in this guide (limits, windows, thresholds, prices) must be read from configuration (`app_settings` table or environment variables), never hard-coded. Product will tune these after launch.

2 The product in one page

Kuchu Puchu is an 18+ dating app for India whose promise is "every profile is a live-verified real person, and your privacy is protected by design". The backend exists to keep that promise.

User journey (and which modules serve it)

Sign in with mobile/email OTP M06 Auth
 → Live selfie (liveness check) M11 Live Verification ← mandatory gate
 → Upload 4–6 photos (face-matched) . M12 Photos
 → Profile, interests, prompts M08, M09
 → Dating preferences M10
 → Discovery cards M18 Discovery
 → Like / pass M17 Interactions
 → Mutual like = match M16 Matches
 → Chat (realtime) M19 Chat
 → Contact exchange / calls (paid) .. M22, M24
 Always available: block, report, unmatched, delete account (M13, M14, M16, M07)
 Optional trust badge: government ID verification (M23)
 Money: Free • Basic ₹199 • Plus ₹499 • Premium ₹999 per month (M21)

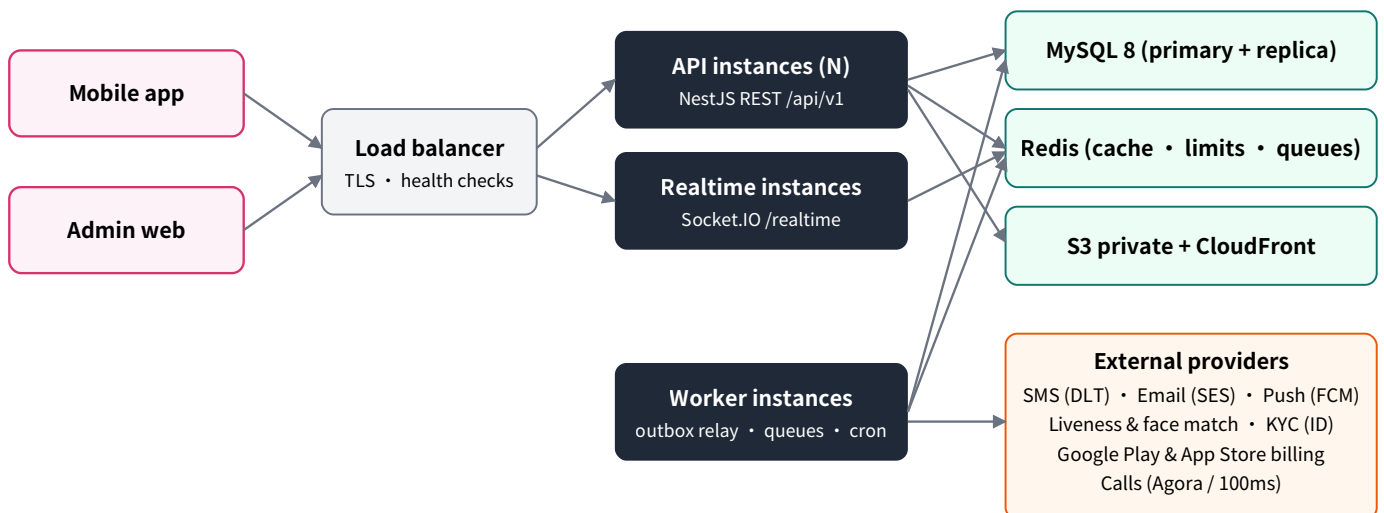
Product principles the code must follow

Principle	What it means for the backend
No fake profiles	No dating API works until the live selfie is approved. Photos must contain the verified face.
Location is never exact	Coordinates are rounded (~110 m) before storage and never returned. Others see only a distance bucket such as "less than 5 km".
Nobody learns by probing	Blocked, hidden and deleted profiles return the same 404. OTP responses are the same for known and unknown users.
Consent before contact	Chat only after a mutual match. Phone numbers are shared only through the contact-exchange flow, and only when the other person accepts.
Safety is never paid	Block, report, unmatch, verification and chatting inside a match are free on every plan.
You own your data	Account deletion removes or anonymises everything. Every table defines what happens on deletion.
Revocation is immediate	Bans, blocks and logouts take effect on the next request or socket event.

3 Architecture

3.1 Style: a modular monolith

One NestJS codebase, split into strict modules. It is deployed as three process types from the **same build**: **API** (REST), **Realtime** (Socket.IO) and **Worker** (background jobs). No microservices at launch. The module boundaries are drawn so that any module can be extracted later if it ever needs its own scaling.



Component	Why it exists
API instances	Stateless. Any instance can serve any request, so we scale by adding instances.
Realtime instances	Hold WebSocket connections. Redis adapter lets an event reach a user on any instance.
Worker instances	Everything slow or retryable: push, email, SMS retries, image work, cleanups, subscription checks.
MySQL 8.4 LTS	Source of truth. InnoDB, utf8mb4. A read replica serves discovery once it becomes the top DB consumer.
Redis	Rate-limit counters, session and entitlement caches, daily quotas, BullMQ job queues, Socket.IO fan-out.
S3 + CloudFront	Private photo and selfie storage. Clients only get short-lived signed URLs.

3.2 Tech stack

Area	Choice
Runtime / language	Node.js LTS, TypeScript (strict mode)
Framework	NestJS (current major), Express adapter
Database / ORM	MySQL 8.4 LTS, Sequelize v6 + sequelize-typescript, Umzug migrations (synchronize always off)
Cache / queues	Redis 7, ioredis , BullMQ, @nestjs/throttler with Redis storage
Realtime	@nestjs/websockets + Socket.IO (WebSocket transport only) + @socket.io/redis-adapter
Auth	@nestjs/jwt , @nestjs/passport , passport-jwt
Images	sharp (decode, strip metadata, re-encode WebP)
AWS	@aws-sdk/client-s3 , presigner, CloudFront signer, Rekognition, SES
Quality	Jest, Supertest, Testcontainers (MySQL, Redis), ESLint/oxlint, Prettier, Swagger/OpenAPI

Area	Choice
Logs	<code>nestjs-pino</code> JSON logs with request id and PII redaction; Sentry for errors

3.3 Layering inside every module

```

controller → HTTP only: route, guards, DTO validation, @CurrentUser(), Swagger docs
service     → business rules, transactions, throws AppException(ErrorCode)
model       → table mapping (sequelize-typescript); small helpers only
dto         → request DTOs (class-validator) and response DTOs built with static fromModel()
adapters    → wrappers around external providers, always behind an interface

```

Hard rules. Controllers never touch models. Models are never returned from an API; always map to a response DTO. `userId` always comes from the token, never from the request body. The service that starts a use case owns the transaction; helper services accept an optional `transaction`. Modules talk to each other only through exported services or events.

3.4 Folder structure

```

backend/
├─ src/
│  ├─ main.ts           API entry point
│  ├─ realtime.ts       Socket.IO entry point
│  ├─ worker.ts         background jobs entry point
│  └─ app.module.ts
│
│  ├─ config/           typed config loaders + env validation
│  ├─ common/           errors, filters, interceptors, guards, decorators, pagination, utils
│  ├─ database/         database module, migrations/, seeders/
│  ├─ infra/            adapters: redis, storage, queue, sms, email, push, face, kyc,
│  │                   store-billing, calls
│  └─ modules/
│     └─ <module>/     <module>.module.ts, controllers/, services/, models/,
│                       dto/, events/, jobs/, *.spec.ts
├─ test/               integration + e2e tests (Testcontainers)
└─ docker/             Dockerfile, docker-compose.yml (MySQL, Redis, LocalStack for S3)

```

3.5 Module dependency rules (no cycles)

- **Blocks** is a leaf that others read (`isBlockedEitherWay`, `excludedUserIds`). It may call Matches to end a match; Matches never imports Blocks.
- **Side effects go through events.** "Notify on match" is an outbox event handled by Notifications, never a direct call into Notifications.
- **Admin** depends on everything; nothing depends on Admin.
- **Account deletion** uses a registry: each module registers an `AccountDeletionHandler`, so Account does not import every module.
- **Entitlements** (M21) exposes one read service used by others. Until M21 is built, a stub returns the Free plan limits.

4 Engineering conventions

4.1 API conventions

Topic	Rule
Base path	<code>/api/v1</code> . A breaking change means a new version path, never a silent change.
Format	JSON, camelCase keys. Database is snake_case (<code>underscored: true</code>).
Auth header	<code>Authorization: Bearer <accessToken></code>
Success body	<code>{ "success": true, "data": ..., "meta": { "nextCursor": ... } }</code> (meta only for lists)
Error body	<code>{ "success": false, "code": "MATCH_NOT_FOUND", "message": "...", "details": {...}, "requestId": "..." }</code>
Status codes	200 OK • 201 created • 202 accepted (async) • 400 validation • 401 not signed in • 403 not allowed • 404 not found (also for hidden/blocked) • 409 state conflict • 422 business rule • 429 limit • 503 provider down
Lists	Keyset cursor only: <code>?cursor=&limit=</code> , default 20, max 50. Never OFFSET on large tables.
Validation	Global <code>ValidationPipe</code> with <code>whitelist</code> , <code>forbidNonWhitelisted</code> , <code>transform</code> . Unknown fields are rejected.
Idempotency	Writes are naturally idempotent where possible (unique keys). Purchases use an <code>Idempotency-Key</code> header. Messages use <code>clientId</code> .
Time	UTC everywhere. ISO-8601 in JSON. The app converts to local time.

Topic	Rule
Docs	Every endpoint has Swagger decorators. The OpenAPI file is the contract for the app and QA teams.
Client headers	X-App-Version , X-Platform , X-Device-Id on every call. Used for force-update and support.

4.2 Database conventions

- Engine InnoDB, charset `utf8mb4`, collation `utf8mb4_0900_ai_ci`. All timestamps `DATETIME(3)` in UTC.
- Primary keys are **UUID v7** stored as `CHAR(36)`. v7 is time-ordered, so inserts stay index-friendly. High-volume append-only logs (`security_events`, `outbox_events`) use `BIGINT AUTO_INCREMENT`.
- Foreign keys are declared for integrity, with `ON DELETE RESTRICT` by default. Deletion is done by services, never by cascades you did not plan.
- Only `users` and `profiles` use soft delete (`deleted_at`). Everything else is hard-deleted or has an explicit status.
- One migration file per change. Migrations are forward-only in production, idempotent, and reviewed like code. Never edit a migration that has run.
- Every query in a hot path must use an index. Add the index in the same migration as the query that needs it. Check with `EXPLAIN` in the pull request.
- Enums that may grow (event types, notification types) are `VARCHAR` validated in code, so adding a value needs no migration.

4.3 Standard columns

Unless a table says otherwise, it has these columns. They are **not repeated** in the entity tables in section 8.

Column	Type	Notes
<code>id</code>	<code>CHAR(36)</code>	UUID v7, primary key, generated in the application
<code>created_at</code>	<code>DATETIME(3)</code>	Set on insert
<code>updated_at</code>	<code>DATETIME(3)</code>	Set on every update

4.4 Code and Git conventions

- Branches: `main` (production), `develop` (staging), `feature/M09-profile-location`. Pull requests need one review and a green pipeline.
- Commit style: `feat(M17): add priority like`, `fix(M06): ....`. The module code in the scope makes history searchable.
- Errors: throw `AppException(ErrorCode.X, details?)`. Never throw raw strings. Every code is listed in Appendix C.
- Logs: structured JSON. Never log OTPs, tokens, phone numbers, emails, message bodies, coordinates or photo URLs.
- Config: add every new variable to the config loader **and** to the env validation **and** to Appendix D.
- External providers: always call them through an adapter interface in `infra/`, with a fake implementation for tests and local development.

5 Security baseline

These rules apply to every module. A pull request that breaks one of them is not merged.

#	Rule
S1	Authorization in the service, not only the route. Every read or write of another user's data goes through one check (for example <code>getActiveForParticipant(matchId, me)</code>). Not allowed returns 404, never 403.
S2	No secrets at rest in plain text. OTPs, refresh tokens and identifier lookups are stored as HMAC-SHA256. Secrets are at least 32 characters and differ from each other.
S3	Session checked on every request. JWT signature, expiry, issuer and audience, then the session row (cached in Redis, cache deleted on revoke). Roles are read from the database, not trusted from the token.
S4	Rate limits everywhere. Global per-IP limit, plus per-user limits on every write endpoint. Stored in Redis so they work across instances.
S5	Never expose coordinates, exact distance, date of birth, email, phone, internal flags, moderation state, or other users' raw timestamps.
S6	Uploads are hostile. Check magic bytes, cap size and pixels, re-encode every image, strip all metadata. Originals are never stored.
S7	Webhooks are verified (signature or JWT) before any processing. Unknown events are logged and ignored.
S8	Money comes from the store, not the app. Every purchase is verified server-to-server with Google Play or Apple before an entitlement is granted.
S9	Admin surface is separate. <code>RolesGuard</code> on every <code>/admin</code> route, shorter admin sessions, IP allowlist in production, and every admin action written to <code>security_events</code> .
S10	Audit everything security-related through <code>SecurityEventsService.record()</code> , which never throws and never stores raw PII.
S11	Encryption: TLS everywhere; RDS, S3 and backups encrypted at rest; S3 buckets private with Block Public Access.
S12	Dependencies: lockfile committed, automated vulnerability scanning in CI, no package added without a reason in the pull request.

Legal note for India. The DPDP Act 2023 applies to this data. Selfies are biometric-adjacent and ID data is sensitive. Store the minimum, record consent (version + time) before capture, and never store Aadhaar numbers or ID document images. The Deployment, Security & Operations Guide covers the compliance checklist.

6 Scalability plan

We build it scalable from day one by following a few design rules, not by adding infrastructure early. The same code runs at every stage below. Only the number of instances and a few settings change.

6.1 Design rules that make it scale

- **Stateless processes.** No user state in memory. Sessions, limits, quotas and caches live in Redis; data in MySQL; files in S3.
- **Keyset pagination only** and **batch loading** (no N+1 queries). Every hot query uses an index.
- **Slow work off the request path.** Push, email, cleanups, exports and provider retries run in workers via the outbox + BullMQ.
- **Idempotent writes** so retries from clients, workers and webhooks are always safe.
- **Per-conversation ordering** via a sequence number locked on one match row, so chat never needs global locks.
- **Cache the hottest reads** in Redis with explicit invalidation: session state, entitlements, block lists, app config.
- **Providers behind adapters** so SMS, face, KYC, push or calls vendors can be swapped without touching business code.

6.2 Growth stages

Stage	Users	Setup	Trigger to move up
Launch	0 – 50k	2 API, 1 realtime, 1 worker, MySQL primary (Multi-AZ), small Redis, S3 + CloudFront	API CPU > 60% or DB CPU > 50% at peak
Growth	50k – 500k	Auto-scale API 2–8 and realtime 2–4, workers per queue, MySQL read replica for discovery and lists	Discovery p95 > 800 ms, messages table > 100M rows
Scale	500k +	Partition or move messages , precomputed discovery candidates in Redis, photo processing on dedicated workers, split realtime into its own service	Measured need only

6.3 Performance budgets (checked in load tests)

Operation	p95 target
Normal reads (profile, matches, messages page)	< 300 ms
Normal writes (like, send message, update profile)	< 500 ms
Discovery feed (10 cards)	< 800 ms
Photo upload with processing	< 3 s
Realtime delivery (send → recipient socket)	< 1 s

7 Build order: all modules at a glance

Build strictly in this order. Each module only depends on modules above it, so every step can be tested on its own. Effort is a rough estimate for one experienced backend developer and includes tests.

#	Module	Phase	Depends on	Tables	APIs	Effort
M01	Core Foundation	0 Foundation	—	0	0	1 wk
M02	Health	0 Foundation	M01	0	2	incl.
M03	Events, Outbox & Jobs	0 Foundation	M01	1	0	0.5 wk
M04	Users	1 Identity	M01	1	0	0.5 wk
M05	Security Audit	1 Identity	M01	1	0	0.5 wk
M06	Auth	1 Identity	M04, M05	2	10	1.5 wk
M07	Account	1 Identity	M04–M06	1	3	1 wk
M08	Catalogue & App Config	2 Profile	M01	3	3	0.5 wk
M09	Profile	2 Profile	M04, M08	3	8	1.5 wk
M10	Preferences	2 Profile	M09	1	2	0.5 wk
M11	Live Verification	2 Profile	M04, M06	1	3	1.5 wk
M12	Photos (Media)	2 Profile	M09, M11	1	4	1.5 wk
M13	Blocks	3 Safety	M04	1	3	0.5 wk

#	Module	Phase	Depends on	Tables	APIs	Effort
M14	Reports	3 Safety	M13, M03	1	2	1 wk
M15	Moderation & Admin	3 Safety (+later)	all earlier	3	18	2 wk
M16	Matches	4 Dating	M04, M13	2	3	1 wk
M17	Interactions	4 Dating	M13, M16, M21 stub	1	4	1 wk
M18	Discovery	4 Dating	M09–M17	0	2	1.5 wk
M19	Chat & Realtime	5 Communication	M16, M13, M03	1	4 + WS	2 wk
M20	Notifications	5 Communication	M03, M06	3	7	1.5 wk
M21	Billing & Entitlements	6 Monetization	M04, M03	5	7	2.5 wk
M22	Contact Exchange	6 Monetization	M16, M19, M21	1	4	1 wk
M23	ID Verification	6 Monetization	M11, M09	1	3	1 wk
M24	Calls	8 Post-launch	M16, M19, M21	1	3	1.5 wk
Total				35	95	~27 wk

Totals: 35 tables, 95 REST endpoints, about 25 working weeks to launch (M01–M23) for one developer, plus about 1.5 weeks for calls (M24). Two backend developers can run Phase 5 and Phase 6 in parallel. Add 2 weeks of hardening (security review, load test, bug fixing) before launch.

8 Module specifications

Phase 0 • Foundation

No product feature yet. This phase gives every later module the same errors, logs, config, database access and background-job system.

M01

Core Foundation

Phase 0

Depends on: —

Tables: 0

APIs: 0

Purpose. The skeleton every other module plugs into: bootstrap, configuration, database, Redis, errors, logging, validation, guards and decorators.

Build list

Item	Details
Bootstrap	Three entry files: <code>main.ts</code> (API), <code>realtime.ts</code> , <code>worker.ts</code> . <code>helmet</code> , compression, CORS allowlist, global prefix <code>/api/v1</code> , body limit 100 kb (multipart routes set their own), graceful shutdown on SIGTERM.
Config	<code>@nestjs/config</code> with typed <code>registerAs</code> loaders. Boot fails fast if any variable is missing or invalid. Secrets ≥ 32 chars. <code>TRUST_PROXY</code> required in production.
Database	Sequelize module, connection pool (min 2, max 20 per instance), Umzug migration runner (<code>npm run migrate</code>), seeders for catalogue data.
Redis	One shared <code>ioredis</code> client module + BullMQ connection. Key naming: <code>kp:<area>:<id></code> .
Errors	<code>ErrorCode</code> enum, <code>AppException</code> , global <code>AllExceptionsFilter</code> producing the error body from section 4.1. Library messages never leak to clients.
Responses	<code>ResponseInterceptor</code> wraps data into the success body.
Logging	pino JSON logs, request id (<code>x-request-id</code>), redaction of auth headers, tokens, OTP, phone, email.
Guards & decorators	<code>JwtAuthGuard</code> , <code>VerifiedUserGuard</code> , <code>RolesGuard</code> + <code>@Roles()</code> , <code>EntitlementGuard</code> + <code>@RequiresEntitlement()</code> , <code>@CurrentUser()</code> , <code>@ClientContext()</code> (IP, UA, app version, device id).
Rate limiting	<code>@nestjs/throttler</code> with Redis storage. Default 100 requests / 60 s per IP. Per-user limits via a custom tracker keyed by user id.
Utilities	UUID v7 generator, text sanitizer (trim, collapse whitespace, strip control chars), cursor encode/decode (base64url JSON), HMAC helpers, distance bucket helper.
Docs & tooling	Swagger at <code>/api/docs</code> (disabled in production), Dockerfile, docker-compose (MySQL, Redis, LocalStack), lint, test and CI pipeline.

Done when: app boots with valid env and refuses invalid env • error and success bodies match 4.1 • CI runs lint, type-check, unit and integration tests • docker-compose up gives a working local stack

M02

Health

Phase 0

Depends on: M01

Tables: 0

APIs: 2

Purpose. Tells the load balancer and monitoring whether an instance is alive and ready.

	Endpoint	Access	What it does
GET	<code>/health</code>	Public	Liveness. Returns 200 if the process is running. No dependencies checked.
GET	<code>/health/ready</code>	Public	Readiness. Pings MySQL and Redis (timeout 1 s). 503 if either fails, so the load balancer stops routing to it.

Done when: both endpoints excluded from rate limits and auth • readiness returns 503 when the DB is down

M03

Events, Outbox & Jobs

Phase 0

Depends on: M01

Tables: 1

APIs: 0

Purpose. Reliable side effects. A service writes an event in the same transaction as its change; a worker delivers it to handlers. Also hosts the scheduler for all periodic jobs.

outbox_events — one row per domain event (uses BIGINT id, no updated_at)

Column	Type	Rules / notes
id	BIGINT UNSIGNED AI	Primary key
event_type	VARCHAR(64)	e.g. <code>match.created</code> (Appendix A)
aggregate_id	CHAR(36)	Id of the main entity (match id, user id...)
payload	JSON	Ids and small values only. No PII copies.
status	ENUM(pending, done, failed)	Default pending
attempts	TINYINT UNSIGNED	Default 0, max 8
available_at	DATETIME(3)	Next time the relay may pick it
last_error	VARCHAR(500) NULL	Last handler error message
created_at	DATETIME(3)	

Indexes: (status, available_at)

How it works

- `OutboxService.publish(type, aggregateId, payload, transaction)` must be called inside the caller's transaction.
- The **relay** (worker) runs every second: `SELECT ... WHERE status='pending' AND available_at<=now ORDER BY id LIMIT 200 FOR UPDATE SKIP LOCKED`, pushes each event into its BullMQ queue, marks it `done`.
- BullMQ handlers retry with exponential backoff (2^n s, max 8). After that the job is failed and an alert fires.
- **Handlers must be idempotent:** they can run twice. Use unique keys (for example `notifications.dedupe_key`).
- Periodic jobs (Appendix B) are BullMQ repeatable jobs, so exactly one worker runs each tick, however many workers exist.
- Done events are deleted after 7 days by a cleanup job.

Done when: an event published in a rolled-back transaction is never delivered • a handler that fails is retried and then marked failed • two workers never process the same event at the same time

Phase 1 • Identity

After this phase a person can sign in with OTP, keep a secure session on their phone, and delete their account.

M04 Users

Phase 1

Depends on: M01 **Tables:** 1 **APIs:** 0

Purpose. Owns the account record: identifiers, role, status and activity timestamps. It has no API of its own; other modules use `UserService`.

users — one row per account (+ standard columns, soft delete)

Column	Type	Rules / notes
email	VARCHAR(254) NULL UNIQUE	Lower-cased. Null after account deletion.
phone	VARCHAR(16) NULL UNIQUE	E.164, e.g. +919812345678. Null after deletion.
email_verified_at	DATETIME(3) NULL	Set on first successful email OTP
phone_verified_at	DATETIME(3) NULL	Set on first successful phone OTP
role	ENUM(user, moderator, admin)	Default user. Changed only by admins.
status	ENUM(active, suspended, banned, deactivated)	One status field. <code>canAuthenticate()</code> = status is active and not deleted.
suspended_until	DATETIME(3) NULL	Set with status suspended; a job lifts it
discovery_restricted_at	DATETIME(3) NULL	Hidden from discovery pending review (M14/M15)
last_login_at	DATETIME(3) NULL	Updated on OTP verify
last_active_at	DATETIME(3) NULL	Updated at most every 5 min (Redis throttle). Used for ranking.
deleted_at	DATETIME(3) NULL	Soft delete

Indexes: email unique, phone unique, (status), (last_active_at)

Service API

- `findById, findByIdentifier(type, value), createVerified(type, value, tx)` (handles the unique-key race by re-reading).
- `setStatus(userId, status, reason, until?)` used only by moderation; it also revokes sessions and disconnects sockets through an event.
- `touchLastActive(userId)` called by a global interceptor, throttled with a Redis key that expires after 5 minutes.

Done when: identifiers normalised before every lookup • concurrent registration with the same phone creates exactly one user

M05 Security Audit

Phase 1

Depends on: M01 **Tables:** 1 **APIs:** 0

Purpose. Append-only log of security and admin events for investigations and abuse detection. Used by every module; viewed through the admin API (M15).

security_events — BIGINT id, no updated_at

Column	Type	Rules / notes
id	BIGINT UNSIGNED AI	
user_id	CHAR(36) NULL	Subject of the event, if any
actor_user_id	CHAR(36) NULL	Who did it (moderator/admin for <code>admin.*</code> events)
event_type	VARCHAR(64)	e.g. <code>auth.login_succeeded</code> , <code>admin.user_banned</code>
ip_address / user_agent	VARCHAR(45) / VARCHAR(255) NULL	
metadata	JSON NULL	Never raw PII, OTPs or tokens. Identifiers only as a 12-char HMAC prefix.
created_at	DATETIME(3)	

Indexes: (user_id, created_at), (event_type, created_at)

- `SecurityEventsService.record()` never throws into the caller; failures are logged.
- Retention: 365 days (job), `admin.*` events 3 years.

Done when: every auth, moderation and admin action records an event • a failing insert never breaks the calling request

M06 Auth

Phase 1

Depends on: M04, M05 **Tables:** 2 **APIs:** 10

Purpose. Passwordless sign-in with a one-time code, secure sessions, token refresh with theft detection, device management and step-up re-authentication for dangerous actions.

otp_verifications — one row per code sent (no updated_at)

Column	Type	Rules / notes
identifier_hash	CHAR(64)	HMAC-SHA256(<code>OTP_HASH_SECRET</code> , "type:normalised"). The identifier itself is never stored here.
channel	ENUM(sms, email)	
purpose	ENUM(login, reauth)	
otp_hash	CHAR(64)	HMAC of the code. Compare in constant time.
attempts	TINYINT UNSIGNED	Atomic <code>attempts = attempts + 1 WHERE attempts < max</code>
expires_at	DATETIME(3)	now + 5 min
consumed_at	DATETIME(3) NULL	Set once, by a conditional update
request_ip	VARCHAR(45)	For the per-IP hourly cap

Indexes: (identifier_hash, created_at), (request_ip, created_at)

sessions — one row per signed-in device

Column	Type	Rules / notes
user_id	CHAR(36) FK users	
refresh_token_hash	CHAR(64)	HMAC of the current refresh secret
previous_token_hash	CHAR(64) NULL	Detects reuse of an old token
device_id	VARCHAR(100)	Per-install id from the app
device_name / platform / app_version	VARCHAR(100) / ENUM(android, ios) / VARCHAR(20)	Shown in the device list
ip_address / user_agent	VARCHAR(45) / VARCHAR(255)	Last seen values
last_used_at	DATETIME(3)	
expires_at	DATETIME(3)	Sliding: now + 30 days on each refresh
absolute_expires_at	DATETIME(3)	created_at + 90 days. Never extended.

Column	Type	Rules / notes
<code>reauthenticated_at</code>	DATETIME(3) NULL	Set by step-up OTP; valid 10 min
<code>revoked_at</code> / <code>revoked_reason</code>	DATETIME(3) NULL / VARCHAR(40)	logout, logout_all, replaced, reuse_detected, restricted, deleted

Indexes: (`user_id`, `revoked_at`), (`user_id`, `device_id`)

Endpoint	Access	What it does
POST <code>/auth/otp/request</code>	Public	Body { <code>channel</code> , <code>identifier</code> }. Always the same 200 response, whether the user exists or not.
POST <code>/auth/otp/verify</code>	Public	Body { <code>channel</code> , <code>identifier</code> , <code>otp</code> , <code>deviceId</code> , <code>deviceName</code> , <code>platform</code> , <code>appVersion</code> }. Creates the user on first login. Returns { <code>accessToken</code> , <code>refreshToken</code> , <code>expiresIn</code> , <code>user</code> , <code>isNewUser</code> , <code>nextStep</code> }.
POST <code>/auth/refresh</code>	Public	Body { <code>refreshToken</code> }. Rotates and returns a new pair.
POST <code>/auth/logout</code>	Public	Body { <code>refreshToken</code> }. Revokes that session. Always 200.
POST <code>/auth/logout-all</code>	User	Revokes every session of the caller, including this one.
GET <code>/auth/me</code>	User	Account summary + <code>nextStep</code> (selfie, photos, profile, preferences, done).
GET <code>/auth/sessions</code>	User	Active devices: name, platform, last used, <code>current</code> : <code>true</code> .
DELETE <code>/auth/sessions/:sessionId</code>	User	Revoke one of the caller's own sessions (404 if not theirs).
POST <code>/auth/reauth/request</code>	User	Sends an OTP to the account's own identifier for step-up.
POST <code>/auth/reauth/verify</code>	User	Body { <code>otp</code> }. Sets <code>reauthenticated_at</code> on the current session.

Rules

Rule	Default
Code	6 digits from a CSPRNG, valid 5 min, 5 attempts, one active code per identifier (issuing a new one expires older ones), consumed once
Resend / caps	60 s cooldown (429 <code>OTP_COOLDOWN</code> + <code>retryAfterSeconds</code>), 5 codes / hour per identifier, 20 / hour per IP (config, because of mobile carrier NAT)
HTTP throttle	request: 5 / min per IP • verify: 10 / min per IP
No enumeration	Same response for unknown, known and banned identifiers. Every verify failure is the same 401 <code>OTP_INVALID</code> .
SMS fraud guard	Country allowlist (+91), daily global SMS budget with alert, DLT-registered template
Access token	JWT HS256, 15 min, claims { <code>sub</code> , <code>sid</code> , <code>iss</code> , <code>aud</code> }. Role is read from the DB.
Refresh token	Opaque < <code>sessionId</code> >.<64-char <code>secret</code> >. Rotated on every use. Reuse of an old token revokes all the user's sessions and sends a security alert.
Device binding	Login with a <code>deviceId</code> that already has a session replaces that session. A new device triggers a <code>auth.new_device</code> event (push + email).
Step-up	Account deletion and data export require <code>reauthenticated_at</code> within 10 min, else 403 <code>REAUTH_REQUIRED</code> .
Onboarding gate	<code>nextStep</code> is computed from state: <code>selfie</code> → <code>photos</code> → <code>profile</code> → <code>preferences</code> → <code>done</code> . <code>VerifiedUserGuard</code> returns 403 <code>ONBOARDING_INCOMPLETE</code> with <code>details.nextStep</code> .

Done when: no plaintext OTP or token anywhere in DB or logs • race tests: two verifies of one code, two refreshes of one token • reuse detection revokes all sessions • banned user gets identical OTP response

M07 Account

Phase 1 (hooks grow with each phase)

Depends on: M04, M05, M06 **Tables:** 1 **APIs:** 3

Purpose. Account-wide actions that cross modules: deletion and data export.

data_export_requests

Column	Type	Rules / notes
<code>user_id</code>	CHAR(36) FK users	
<code>status</code>	ENUM(pending, processing, ready, expired, failed)	
<code>file_key</code>	VARCHAR(200) NULL	S3 key of the ZIP (private)
<code>expires_at</code>	DATETIME(3) NULL	Download link valid 24 h
<code>completed_at</code>	DATETIME(3) NULL	

Indexes: (`user_id`, `created_at`)

Endpoint	Access	What it does
DELETE /account	User	Deletes the account (step-up required). Response warns that store subscriptions must be cancelled in Google Play / App Store.
POST /account/export	User	Starts an export (step-up required, 1 per 24 h). 202.
GET /account/export	User	Latest export status and a signed download URL when ready.

Deletion design

Each module registers an `AccountDeletionHandler { order, handle(userId, tx) }`. `AccountService.delete()` locks the user row, runs all handlers in **one transaction**, then commits. File purges run after commit through outbox events.

Module	What happens on deletion
Auth / Notifications	All sessions revoked, push devices deleted, sockets disconnected, notifications and preferences deleted
Profile / Preferences	Profile fields scrubbed and soft-deleted; interests, prompts and preferences deleted
Photos / Verification	Photos marked deleted and S3 objects purged; selfie and ID verification rows anonymised, selfie object deleted
Interactions / Matches / Chat	Likes and passes deleted both ways; matches closed (account_deleted); own messages scrubbed to tombstones
Blocks / Reports	Blocks deleted both ways; reports filed by the user anonymised; reports against the user kept for the retention window
Billing / Contacts / Calls	Subscriptions marked cancelled locally; contact exchanges deleted; call rows anonymised
Users	email and phone set to null, status deactivated, soft-deleted. Audit event stores identifier hashes only. If the user was banned, hashes go to ban_hashes .

Done when: one failing handler rolls back the whole deletion • same phone can register again and gets a new user id • export contains own data only

Phase 2 • Profile & live verification

After this phase a signed-in person passes the live selfie, uploads face-matched photos, and completes a profile and preferences.

M08 Catalogue & App Config	Phase 2
Depends on: M01 Tables: 3 APIs: 3	

Purpose. Read-mostly reference data (interests, prompt questions) and runtime settings (feature flags, force-update version, maintenance mode, all tunable limits).

interests — seeded catalogue

Column	Type	Rules / notes
name / slug	VARCHAR(50) / VARCHAR(50) UNIQUE	e.g. Music / music
category / icon	VARCHAR(30) / VARCHAR(50)	
is_active / sort_order	BOOLEAN / SMALLINT	Inactive items hidden, never deleted

prompts — profile question catalogue

Column	Type	Rules / notes
text	VARCHAR(150)	e.g. "My perfect Sunday is..."
category	VARCHAR(30)	
is_active / sort_order	BOOLEAN / SMALLINT	

app_settings — key-value runtime config (no id; key is the PK)

Column	Type	Rules / notes
key	VARCHAR(64) PK	e.g. likes.daily.free, app.min_version.android
value	JSON	Validated by a schema per key
updated_by	CHAR(36) NULL	Admin who changed it
updated_at	DATETIME(3)	

Endpoint	Access	What it does
GET /catalog/interests	User	Active interests grouped by category
GET /catalog/prompts	User	Active prompt questions
GET /app/config	Public	Min app version per platform, maintenance flag, public feature flags, support links

- All three are cached in Redis for 10 min and invalidated when an admin changes a setting (M15).

- `SettingsService.get<T>(key, default)` is the only way code reads tunable limits.

Done when: seeders create interests and prompts • changing a setting takes effect within one cache refresh

M09 Profile

Phase 2

Depends on: M04, M08 **Tables:** 3 **APIs:** 8

Purpose. The user's dating profile, its completion score, and the public view other users are allowed to see.

profiles — one per user (+ standard columns, soft delete)

Column	Type	Rules / notes
<code>user_id</code>	CHAR(36) FK UNIQUE	
<code>display_name</code>	VARCHAR(50)	2–50 chars, letters, spaces, ' . - . Re-moderated on change.
<code>date_of_birth</code>	DATE	Age 18–100. Immutable, except when overwritten by ID verification.
<code>dob_source</code>	ENUM(self, id_verified)	Default self
<code>gender</code>	ENUM(woman, man, non_binary, other)	Max 2 changes per 30 days
<code>show_gender</code>	BOOLEAN	Default true
<code>bio</code>	VARCHAR(500) NULL	Sanitised. Contact details are rejected (see rules).
<code>occupation / education</code>	VARCHAR(100) NULL	
<code>relationship_intent</code>	ENUM(long_term, short_term, casual, marriage, friends, unsure)	
<code>height_cm / languages</code>	SMALLINT NULL / JSON NULL	Optional
<code>latitude / longitude</code>	DECIMAL(6,3) / DECIMAL(7,3) NULL	Rounded to 3 decimals (~110 m) before saving. Never returned.
<code>geohash</code>	CHAR(6) NULL	Derived from the rounded point, for fast area queries
<code>city / state / country</code>	VARCHAR(100) / VARCHAR(100) / CHAR(2)	Only city is shown to others
<code>location_updated_at</code>	DATETIME(3) NULL	Updates allowed every 15 min at most
<code>is_discoverable</code>	BOOLEAN	False = snooze (hidden from discovery, matches continue)
<code>visibility</code>	ENUM(public, hidden)	Hidden = profile 404 to everyone else
<code>incognito</code>	BOOLEAN	Premium: only people you liked can see you
<code>completion</code>	TINYINT UNSIGNED	0–100, computed by the server
<code>approved_photo_count</code>	TINYINT UNSIGNED	Kept in sync by M12
<code>face_verified_at / id_verified_at</code>	DATETIME(3) NULL	Set by M11 / M23
<code>gender_changed_at / gender_change_count</code>	DATETIME(3) NULL / TINYINT	For the change limit
<code>deleted_at</code>	DATETIME(3) NULL	

Indexes: `user_id` unique, (`is_discoverable`, `visibility`, `deleted_at`), (`geohash`), (`date_of_birth`), (`gender`)

profile_interests — composite PK, no id

Column	Type	Rules / notes
<code>profile_id / interest_id</code>	CHAR(36) FK / CHAR(36) FK	PK (<code>profile_id</code> , <code>interest_id</code>). Max 10 per profile.
<code>created_at</code>	DATETIME(3)	

Indexes: (`interest_id`) for shared-interest counts

profile_prompts — up to 3 answers

Column	Type	Rules / notes
<code>profile_id / prompt_id</code>	CHAR(36) FK / CHAR(36) FK	Unique (<code>profile_id</code> , <code>prompt_id</code>)
<code>answer</code>	VARCHAR(200)	Sanitised, contact details rejected
<code>position</code>	TINYINT	0–2, unique per profile

Endpoint	Access	What it does
GET <code>/profile</code>	User	Own full profile (never raw coordinates, only <code>hasLocation</code>)
POST <code>/profile</code>	User	Create (after selfie approved). 409 if one exists.
PATCH <code>/profile</code>	User	Partial update. <code>dateOfBirth</code> is rejected.

Endpoint	Access	What it does
PUT /profile/location	User	Body {lat, lng}. Rounded, reverse-geocoded to city. Ignored if < 15 min since last.
GET /profile/completion	User	{score, missingFields[]}
PUT /profile/interests	User	Replace all selections atomically (max 10)
PUT /profile/prompts	User	Replace prompt answers (max 3)
GET /profiles/:userId	Verified	Public view of another user. Silent 404 if not allowed.

Rules

- **Completion weights** (sum 100): photos ≥ 4 : 25 • bio 15 • prompts ≥ 1 : 10 • interests ≥ 3 : 10 • display name 5 • DOB 5 • gender 5 • location 10 • occupation 5 • education 5 • ID badge 5.
- **Public view** returns only: userId, displayName, age, gender (if shown), bio, occupation, education, city, distance bucket, shared-flagged interests, prompts, photos (signed URLs), faceVerified, idVerified, activity bucket.
- **Distance buckets**: LT_2_KM, LT_5_KM, LT_10_KM, LT_25_KM, LT_50_KM, LT_100_KM, GT_100_KM. **Activity**: ACTIVE_TODAY, ACTIVE_THIS_WEEK, or omitted.
- **Access to /profiles/:userId**: allowed if there is an active match, or the target is currently eligible for the viewer in discovery. Otherwise 404 PROFILE_NOT_FOUND, same body for every reason.
- **No contact details in text fields**. Bio, prompts and name are checked for phone numbers (including spaced or spelled-out digits), emails, UPI ids and social handles. Rejected with 422 CONTACT_DETAILS_NOT_ALLOWED. This protects the paid contact-exchange feature and users' safety.
- Rate limit: 30 profile updates per hour per user.

Done when: age under 18 rejected at create • coordinates never appear in any response • public view returns identical 404 for blocked, hidden and missing users

M10 Preferences

Phase 2

Depends on: M09 **Tables:** 1 **APIs:** 2

Purpose. Who the user wants to see. Discovery applies these filters in both directions.

dating_preferences — one per user

Column	Type	Rules / notes
user_id	CHAR(36) FK UNIQUE	
min_age / max_age	TINYINT UNSIGNED	$18 \leq \min \leq \max \leq 100$, gap ≥ 2
preferred_genders	JSON	Non-empty array of gender values
max_distance_km	SMALLINT UNSIGNED	1–200 (config)
intents	JSON NULL	Advanced filter (paid): relationship intents
verified_only	BOOLEAN	Advanced filter (paid): only ID-verified profiles
advanced	JSON NULL	Paid: education levels, languages, height range

Indexes: user_id unique; multi-valued index on preferred_genders (MySQL 8.0.17+)

Endpoint	Access	What it does
GET /preferences	User	Own preferences with isConfigured flag
PUT /preferences	User	Create or replace. Paid fields need the advanced_filters entitlement (403 ENTITLEMENT_REQUIRED).

- If a plan lapses, stored advanced filters are kept but ignored by discovery until the entitlement returns.

Done when: invalid ranges rejected with clear messages • advanced fields blocked on the Free plan

M11 Live Verification (selfie gate)

Phase 2

Depends on: M04, M06 **Tables:** 1 **APIs:** 3

Purpose. Proves a real, live person owns the account before any dating feature opens. The approved selfie becomes the face reference for all photos.

face_verifications — one row per attempt

Column	Type	Rules / notes
<code>user_id</code>	CHAR(36) FK users	
<code>provider / provider_session_id</code>	VARCHAR(30) / VARCHAR(100)	e.g. rekognition-liveness, hyperverge
<code>status</code>	ENUM(created, processing, approved, pending_review, rejected, expired)	
<code>liveness_score</code>	DECIMAL(5,2) NULL	From the provider, fetched server-side
<code>selfie_key</code>	VARCHAR(200) NULL	S3 <code>v/<id>.webp</code> , private
<code>rejection_reason</code>	VARCHAR(40) NULL	not_live, face_not_visible, poor_quality, duplicate_suspected
<code>reviewer_id / reviewed_at</code>	CHAR(36) NULL / DATETIME(3) NULL	Set for human decisions
<code>consent_version / consented_at</code>	VARCHAR(20) / DATETIME(3)	Consent screen shown before capture
<code>decided_at</code>	DATETIME(3) NULL	

Indexes: (`user_id`, `created_at`), (`status`, `created_at`) for the review queue

Endpoint	Access	What it does
GET <code>/verification</code>	User	Face status (and ID status once M23 exists), attempts left
POST <code>/verification/face/session</code>	User	Body <code>{consentVersion}</code> . Creates a provider liveness session, returns the SDK session token.
POST <code>/verification/face/complete</code>	User	Body <code>{sessionId}</code> . Server fetches the result from the provider (never trusts the app) and decides.

Decision rules (thresholds in config)

Result	Action
Liveness ≥ 90 and one clear face	Auto-approve: set <code>profiles.face_verified_at</code> (or pending until the profile exists), publish <code>verification.face.approved</code>
Liveness 70–90, or quality issues	<code>pending_review</code> → moderator queue (M15), target decision within 2 hours
Liveness < 70 or no face	<code>rejected</code> with a reason the user can act on

- Attempts: 3 per 24 h, 10 in total, then 429 `VERIFICATION_ATTEMPTS_EXCEEDED` and a support path.
- The selfie is **never shown** to other users. It is kept while the account is active (reference for photo matching) and deleted on account deletion or 30 days after a rejection.
- No face templates or face collections are stored. Face comparison calls are stateless.
- Re-verification can be required by moderation (sets status back to required, `nextStep = selfie`).

Done when: no dating endpoint reachable without approval • the app cannot fake a pass (result always fetched server-side) • consent version stored on every attempt

M12 Photos (Media)

Phase 2

Depends on: M09, M11 (face reference) **Tables:** 1 **APIs:** 4

Purpose. Profile photo upload, safe processing, face matching against the live selfie, moderation, ordering and private delivery.

photos

Column	Type	Rules / notes
<code>user_id</code>	CHAR(36) FK users	
<code>position</code>	TINYINT UNSIGNED	0 = primary. Contiguous 0...n-1.
<code>status</code>	ENUM(processing, approved, pending_review, rejected, deleted)	Only approved photos are shown to others
<code>storage_prefix</code>	VARCHAR(100)	<code>p/<photoId>/</code> (no user id in keys)
<code>width / height</code>	SMALLINT UNSIGNED	Of the large rendition
<code>content_hash</code>	CHAR(64)	SHA-256 of the original bytes (re-upload of banned photos)
<code>face_match_score</code>	DECIMAL(5,2) NULL	Similarity to the verified selfie
<code>moderation_labels</code>	JSON NULL	Labels from the moderation provider
<code>rejection_reason</code>	VARCHAR(40) NULL	explicit, face_mismatch, no_face, low_quality, duplicate_banned
<code>deleted_at / purged_at</code>	DATETIME(3) NULL	Purged = S3 objects removed

Indexes: (`user_id`, `status`, `position`), (`content_hash`)

	Endpoint	Access	What it does
GET	/profile/photos	User	Own photos (any status) with signed URLs
POST	/profile/photos	User	Multipart field file , optional position . 201 with status.
PUT	/profile/photos/order	User	Body {photoIds[]} . Must be exactly the caller's photo set.
DELETE	/profile/photos/:photoId	User	Soft-delete, compact positions, purge S3 after commit

Upload pipeline

```

1 limits: max 6 photos (409 PHOTO_LIMIT_REACHED), ≤ 10 MB, throttle 20 uploads / hour
2 magic bytes must be JPEG / PNG / WebP (400 PHOTO_INVALID_FILE). HEIC is converted by the app.
3 sharp: ≥ 400×400, ≤ 50 MP, auto-rotate, strip ALL metadata, 3 WebP sizes: 160 / 640 / 1280 px
4 sha256 → reject if in ban_hashes (photo)
5 moderation scan (explicit → rejected, borderline → pending_review)
6 face match vs verified selfie (CompareFaces): ≥ 90 match, 70–90 pending_review, no face → allowed only as non-
  primary
7 PutObject x3 to the private bucket (SSE)
8 TX: insert photo, shift positions, update approved_photo_count + completion, publish photo.* event

```

Rules

- **Discovery needs at least 4 approved photos** (**PHOTOS_MIN_FOR_DISCOVERY**), and the **primary photo plus at least 2 others must contain the verified face**.
- Delivery: private bucket + CloudFront signed URLs valid 1 hour, only generated for viewers allowed to see the profile. Clients cache by **photoId + size**, not by URL.
- Changing the primary photo to one without a verified face match is refused (422 **PHOTO_FACE_MISMATCH**).
- Photos are uploaded right after the selfie, before the profile exists. **approved_photo_count** and completion are computed when the profile is created and kept in sync afterwards.
- Scale path: when upload p95 > 3 s or sharp uses > 20% API CPU, switch to presigned direct upload + worker processing (same table, status **processing**).

Done when: EXIF/GPS never present in stored files (test with a geotagged image) • pending and rejected photos visible only to the owner • deleting a photo removes S3 objects within minutes

Phase 3 • Safety & moderation

Built **before** discovery: users must be able to block and report from the first moment they can see anyone.

M13	Blocks	Phase 3
Depends on: M04 (M16 hook added later) Tables: 1 APIs: 3		

Purpose. Instant, silent, two-way separation between two users.

blocks

Column	Type	Rules / notes
blocker_user_id	CHAR(36) FK users	
blocked_user_id	CHAR(36) FK users	
display_snapshot	JSON	Name + thumbnail key captured at block time, for the blocked list

Indexes: unique (**blocker_user_id**, **blocked_user_id**), (**blocked_user_id**)

	Endpoint	Access	What it does
POST	/blocks	User	Body {userId} . 200 even if the user does not exist (no oracle). Idempotent.
DELETE	/blocks/:userId	User	Unblock. Does not restore an ended match.
GET	/blocks	User	Users I blocked (cursor list)

Rules

- Effect in both directions: removed from each other's discovery, likes and profile views return 404, any active match ends with status **blocked** in the **same transaction** (hook registered by M16).
- The blocked user is never told. Their app just receives **match:ended** with no reason.
- **BlocksService.excludedUserIds(userId)** returns both directions. Cached as a Redis set per user, invalidated on block/unblock. This is the most-called safety query.

- Limit: 100 blocks per day per user.

Done when: blocked pair never sees each other anywhere • the response to the blocked user is identical to "user not found"

M14 Reports

Phase 3

Depends on: M13, M03 **Tables:** 1 **APIs:** 2

Purpose. Lets users report a person with evidence captured at that moment, and feeds the moderation queue.

reports

Column	Type	Rules / notes
reporter_user_id	CHAR(36) NULL	Null after reporter deletes account
reported_user_id	CHAR(36) FK users	
category	ENUM(underage, threat_or_violence, offline_harm, sexual_harassment, explicit_content, scam_or_spam, fake_profile, impersonation, hate_speech, harassment, other)	
context_type / context_id	ENUM(profile, photo, message, match, offline) / CHAR(36) NULL	Id required for photo and message
description	VARCHAR(1000) NULL	Sanitised
evidence_snapshot	JSON	Profile fields, photo keys (copied to <code>evidence/<reportId>/</code>), reported message + 20 around it
priority	ENUM(P0, P1, P2)	From category
status	ENUM(pending, under_review, action_taken, dismissed, escalated)	
assigned_to	CHAR(36) NULL	Moderator
resolution_note	VARCHAR(1000) NULL	Internal only
resolved_at	DATETIME(3) NULL	

Indexes: (status, priority, created_at), (reported_user_id, status), (reporter_user_id, created_at)

Endpoint	Access	What it does
POST /reports	User	Body {reportedUserId, category, contextType, contextId?, description?, alsoBlock=true}. 201, or 200 when merged into an open report.
GET /reports	User	My reports with status only (never the action taken)

Rules

Rule	Detail
Priority / SLA	P0 (underage, threats, offline harm): 1 h • P1 (harassment, explicit, scam, fake, impersonation, hate): 12–24 h • P2 (other): 72 h
Valid target	The reporter must have been able to see the context, else 404 <code>REPORT_TARGET_INVALID</code>
De-duplication	Same reporter + same target + open report → merge evidence, return the same id
Auto-block	<code>alsoBlock</code> defaults to true and runs in the same transaction
Auto-restrict	3 distinct established reporters (account ≥ 7 days) in 24 h, or any P0 → set <code>users.discovery_restricted_at</code> until reviewed
Abuse of reports	New accounts count 0.25. Reporters with > 80% dismissed (≥ 10 reports) get lowest queue priority. 10 reports / day per user.

Done when: evidence survives the reported user deleting their messages • P0 report pages the on-call admin via an event

M15 Moderation & Admin

Phase 3 (core) • grows in later phases

Depends on: all earlier modules **Tables:** 3 **APIs:** 18

Purpose. Everything moderators and admins do: review queues, enforcement actions, appeals, audit log and settings. The admin web app is a separate frontend that uses only these APIs.

moderation_actions

Column	Type	Rules / notes
target_user_id / moderator_id	CHAR(36) FK / CHAR(36) FK	
report_id	CHAR(36) NULL	
action	ENUM(warn, remove_content, restrict_discovery, suspend, ban, lift)	
reason_code	VARCHAR(40)	Shown to the user as a category only
note	VARCHAR(1000) NULL	Internal
expires_at	DATETIME(3) NULL	For suspend
reversed_at	DATETIME(3) NULL	Set if overturned on appeal

Indexes: (target_user_id, created_at)

appeals

Column	Type	Rules / notes
moderation_action_id / user_id	CHAR(36) FK / CHAR(36) FK	One appeal per action
message	VARCHAR(1000)	
status	ENUM(pending, upheld, overturned)	
reviewer_id / decided_at	CHAR(36) NULL / DATETIME(3) NULL	Must differ from the original moderator

ban_hashes

Column	Type	Rules / notes
hash_type	ENUM(identifier, photo, device)	
hash	CHAR(64)	HMAC / SHA-256. Unique with hash_type.
source_user_id	CHAR(36) NULL	

Indexes: unique (hash_type, hash)

	Endpoint	Access	What it does
GET	/admin/dashboard	Mod	Queue sizes, SLA breaches, signups, verifications today
GET	/admin/users	Mod	Search by user id, phone or email (exact match only; results masked for moderators)
GET	/admin/users/:userId	Mod	Profile, photos, verification, reports, actions, recent security events
POST	/admin/users/:userId/actions	Mod	Warn, remove content, restrict, suspend, ban, lift. Ban and lift-ban: Admin only.
GET	/admin/reports	Mod	Queue filtered by status and priority, ordered by priority then age
POST	/admin/reports/:reportId/claim	Mod	Assign to me, status under_review
POST	/admin/reports/:reportId/resolve	Mod	Dismiss, take action (links a moderation action) or escalate
GET	/admin/verifications/face	Mod	Selfies waiting for human review
POST	/admin/verifications/face/:id/decision	Mod	Approve or reject with a reason
GET	/admin/photos	Mod	Photos in pending_review
POST	/admin/photos/:photoId/decision	Mod	Approve or reject with a reason
GET	/admin/appeals	Mod	Pending appeals
POST	/admin/appeals/:appealId/decision	Mod	Uphold or overturn (overturn reverses the action)
GET	/admin/audit-log	Admin	security_events filtered by user, type, date
GET	/admin/billing/overview	Admin	Active subscriptions by plan, revenue, refunds (after M21)
PUT	/admin/settings/:key	Admin	Change an app_settings value (validated, audited)
POST	/account/appeals	User	Appeal a suspend or ban. Allowed with a special short-lived appeal token.
GET	/account/appeals	User	My appeals and their status

Action effects

Action	Effect
warn	In-app notice only
remove_content	Photo → rejected, text field cleared, or message → removed_by_moderation
restrict_discovery	Hidden from discovery. Existing matches and chat continue.
suspend	Status suspended until a date, sessions revoked, sockets closed, matches kept

Action	Effect
ban	Status banned, sessions revoked, matches closed, identifier/photo/device hashes added to <code>ban_hashes</code>

- Admin sessions last 8 hours; production admin routes are IP-allowlisted. Every action writes an `admin.*` security event with the actor.
- Moderators see phone and email masked. Only admins see full identifiers, and every reveal is audited.

Done when: a moderator cannot ban • every admin action appears in the audit log • suspension is lifted automatically by a job

Phase 4 • Dating core

After this phase verified users see each other, like, and match.

M16 Matches	Phase 4
Depends on: M04, M13 Tables: 2 APIs: 3	

Purpose. A match is a mutual like and is also the conversation container. It owns the lifecycle and the per-user state.

matches

Column	Type	Rules / notes
<code>user_low_id / user_high_id</code>	CHAR(36) FK / CHAR(36) FK	Sorted pair (low < high). Unique: one match per pair, ever.
<code>status</code>	ENUM(active, unmatched, blocked, closed)	All end states are final. No re-match.
<code>ended_at / ended_by_user_id</code>	DATETIME(3) NULL / CHAR(36) NULL	
<code>end_reason</code>	VARCHAR(30) NULL	unmatched, blocked, account_deleted, account_banned, moderation
<code>last_message_seq</code>	INT UNSIGNED	Default 0. Locked and incremented per message (M19).
<code>last_message_at</code>	DATETIME(3) NULL	

Indexes: unique (`user_low_id`, `user_high_id`), (`status`)

match_participants — two rows per match, composite PK

Column	Type	Rules / notes
<code>match_id / user_id</code>	CHAR(36) FK / CHAR(36) FK	PK (<code>match_id</code> , <code>user_id</code>)
<code>last_read_seq</code>	INT UNSIGNED	Unread = <code>last_message_seq</code> – <code>last_read_seq</code>
<code>last_activity_at</code>	DATETIME(3)	Copy of the last message time, for fast per-user lists
<code>muted_until</code>	DATETIME(3) NULL	Mute notifications for this match
<code>is_active</code>	BOOLEAN	False once the match ends

Indexes: (`user_id`, `is_active`, `last_activity_at`)

Endpoint	Access	What it does
GET <code>/matches</code>	Verified	My active matches, newest activity first (cursor). Each item: counterpart compact card, last message preview, unread count, <code>isNew</code> .
GET <code>/matches/:matchId</code>	Verified	One match. 404 <code>MATCH_NOT_FOUND</code> if not a participant or ended.
DELETE <code>/matches/:matchId</code>	Verified	Unmatch. Silent to the other person (realtime removal only). Idempotent.

- `MatchingService.getActiveForParticipant(matchId, userId)` is the single authorisation check for every match-scoped route (chat, contacts, calls).
- Hooks: `endForBlock(a, b, tx)` for M13, `closeAllForUser(userId, reason, tx)` for deletion and bans.
- Messages of ended matches are hidden immediately and hard-deleted after 90 days, unless referenced by a report.

Done when: non-participants always get 404 • ended match disappears from both lists instantly

M17 Interactions (Like / Pass)	Phase 4
Depends on: M13, M16, M21 (stub) Tables: 1 APIs: 4	

Purpose. Records likes and passes, enforces plan quotas, and creates a match when a like is mutual.

interactions — one row per ordered pair

Column	Type	Rules / notes
<code>actor_user_id / target_user_id</code>	CHAR(36) FK / CHAR(36) FK	Unique pair
<code>type</code>	ENUM(like, pass)	Latest decision wins (rules below)
<code>is_priority</code>	BOOLEAN	Priority like: shown first to the target
<code>note</code>	VARCHAR(140) NULL	Optional note with a priority like, contact details rejected

Indexes: unique (`actor_user_id`, `target_user_id`), (`target_user_id`, `type`, `created_at`)

Endpoint	Access	What it does
POST <code>/likes</code>	Verified	Body <code>{targetUserId, priority?, note?}</code> . Returns <code>{liked: true, match: MatchSummary null}</code> .
POST <code>/passes</code>	Verified	Body <code>{targetUserId}</code> . Returns <code>{passed: true}</code> .
DELETE <code>/passes/:targetUserId</code>	Verified	Undo the most recent pass within 5 min. Needs <code>undo_pass</code> entitlement.
GET <code>/likes/received</code>	Verified	Free/Basic: count + blurred previews. Plus/Premium: full list (cursor).

Rules

- Transitions: none → like/pass • pass → like • like → pass is refused (409 `INTERACTION_ALREADY_LIKED`) • repeating the same action is a no-op 200.
- Target must be a visible, verified user not blocked either way, else 404 `USER_NOT_FOUND`. Liking yourself: 400.
- Match creation (commit-then-check):** commit the like, then check for the reverse like; if found, insert the match. The unique pair key on `matches` makes a double insert safe. No row locks, no deadlocks.
- A reconciliation job every 10 min creates any match missed because of a crash between the two steps.
- Quotas from entitlements (Redis daily counters, reset at midnight IST): likes per day 25 / 100 / unlimited / unlimited; priority likes 0 / 1 per week / 3 per week / 1 per day. Over quota: 429 `LIKE_LIMIT_REACHED` with `retryAfterSeconds`.
- Abuse: 30 actions / min; first 24 h of an account max 50 likes; > 95% likes over 200 actions in 24 h raises `abuse.like_ratio_anomaly`.
- Events: `like.created` (aggregated "someone likes you" push, max 1 per 6 h), `match.created`.

Done when: simultaneous mutual likes create exactly one match • quota enforced per plan and resets daily

M18 Discovery

Phase 4

Depends on: M09–M12, M13, M14, M17, M16, M21 (stub) **Tables:** 0 **APIs:** 2

Purpose. Builds the card feed: strict eligibility filters in both directions, then an explainable ranking.

Endpoint	Access	What it does
GET <code>/discovery</code>	Verified	Query <code>limit</code> (≤ 20), <code>exclude</code> (ids still in the app's queue, ≤ 50). Returns public profile cards and <code>hasMore</code> .
PUT <code>/discovery/travel-location</code>	Verified	Body <code>{lat, lng}</code> or <code>null</code> to clear. Needs <code>travel_mode</code> entitlement.

Viewer must be ready

Otherwise 409 `DISCOVERY_NOT_READY` with `details.missing` ⊂ [selfie, photos, profile, location, preferences].

Eligibility filters (all must pass)

#	Filter
F1–F3	Not self • candidate status active, not deleted, not discovery-restricted • profile discoverable, public, not deleted
F4	Candidate face-verified and has ≥ 4 approved photos
F5–F6	Candidate age in viewer's range and viewer age in candidate's range
F7–F8	Candidate gender in viewer's list and viewer gender in candidate's list
F9–F10	Distance $\leq$ viewer's max and $\leq$ candidate's max (geohash cells, then exact sphere distance)
F11	No block in either direction
F12–F14	Viewer has not liked the candidate; has not passed them in the last 30 days
F15	No match row of any status between the two
F16	Incognito candidates only if they have liked the viewer
F17	Paid advanced filters (intent, verified-only, education, language, height) if the viewer has the entitlement

Ranking (weights in config)

Signal	Weight	Meaning
likedViewer	30	Candidate already liked the viewer (never revealed on the card)
activity	20	Active < 24 h = 1.0, < 3 d = 0.6, < 7 d = 0.3
sharedInterests	15	min(shared, 5) / 5
intentMatch	10	Same intent = 1, either unsure = 0.5
idVerified	10	Has the ID badge
completeness	10	completion / 100
proximity	5	1 – distance / max distance
boost	+25	Candidate has an active boost (M21)
jitter	≤ 3	Stable per viewer, candidate and day (not random per request)

- **Two stages:** one SQL query applies F1–F17 and returns up to 200 ids by recent activity; the app layer then scores, picks the top **limit**, and batch-loads photos and interests.
- The ranker sits behind a **CandidateRanker** interface so a future ML ranker can replace it without touching the endpoint.
- Limits: 30 requests / min, 500 cards / day per user (anti-scraping).
- The viewer's location is always the stored rounded value, never a location sent with the request.
- Log **discovery.card_served** impressions (no PII) for future ranking work.

Done when: every filter covered by an integration test with two-sided data • p95 < 800 ms with 100k seeded profiles in one city

Phase 5 • Communication

After this phase matched users chat in real time and get notifications.

M19	Chat & Realtime	Phase 5
Depends on: M16, M13, M03 (contact rule completed in M22) Tables: 1 APIs: 4 + WS		

Purpose. Messages inside active matches with strict ordering, idempotent sends, read state, typing, and contact-detail protection.

messages

Column	Type	Rules / notes
match_id	CHAR(36) FK matches	
seq	INT UNSIGNED	1, 2, 3... per match. Unique with match_id.
sender_user_id	CHAR(36) FK users	
client_message_id	CHAR(36)	From the app. Unique with match and sender (idempotent retries).
type	ENUM(text, system, contact_card)	contact_card is written only by M22
body	VARCHAR(2000) NULL	Sanitised. Null for tombstones.
status	ENUM(visible, deleted_by_sender, removed_by_moderation)	
flags	JSON NULL	Soft flags for moderation (link, scam keyword...)
deleted_at	DATETIME(3) NULL	

Indexes: unique (match_id, seq), unique (match_id, sender_user_id, client_message_id)

Endpoint	Access	What it does
GET /matches/:matchId/messages	Verified	?beforeSeq=&limit=30 (history, newest first) or ?afterSeq=&limit=100 (re-sync, oldest first)
POST /matches/:matchId/messages	Verified	Body {clientMessageId, type: "text", body}. 201, or 200 with the original on retry.
POST /matches/:matchId/read	Verified	Body {lastReadSeq}. Never moves backwards.
DELETE /matches/:matchId/messages/:messageId	Verified	Unsend own message within 15 min (tombstone keeps seq)

Send flow

```

1  getActiveForParticipant → 404; blocked either way → 404
2  rate limits: 20 / min, 500 / day; max 3 messages before the first reply (429 MESSAGE_AWAITING_REPLY)
3  content: empty → 422; contact details while no accepted contact exchange → 422 CONTACT_SHARING_LOCKED
4  TX: if clientMessageId exists → return it
      UPDATE matches SET last_message_seq = last_message_seq + 1 ... WHERE id=? AND status='active' (row lock =
      ordering)

```

```
INSERT message (seq); sender's last_read_seq = seq; update both participants' last_activity_at
outbox: message.created
5 worker: emit message:new to room user:<recipient>; push if no foreground socket
```

Realtime (Socket.IO, namespace /realtime, WebSocket transport only)

- Connect with `auth: {token}`; same checks as the REST guard; join room `user:<id>`. Session re-checked every 5 min; revoked → disconnect `auth_expired`.
- Server → client: `message:new`, `message:deleted`, `match:new`, `match:ended`, `match:read`, `typing`, `contact:updated`, `call:incoming`. Client → server: `typing` only (max every 3 s). All writes use REST.
- REST is the source of truth: a gap in `seq` on the app triggers an `afterSeq` fetch. Redis adapter delivers across instances.
- No live online status (stalking risk). Read receipts can be switched off by the user.
- Contact detection normalises text first (digits spelled out, spaces, symbols), then checks phone numbers (libphonenumber), emails, UPI ids and social handles.
- Links from accounts younger than 24 h are rejected; links render as plain text with an "open link" warning in the app.
- The same text sent to ≥ 5 matches in 1 h raises `abuse.message_broadcast`.
- Scale: keyset reads on (`match_id`, `seq`); partition or move the table at ~100M rows.

Done when: concurrent sends get unique, gap-free seq • retry with the same `clientMessageId` never duplicates • blocked user cannot send via REST or socket

M20 Notifications

Phase 5

Depends on: M03, M06 (sessions) **Tables:** 3 **APIs:** 7

Purpose. Push notifications, in-app inbox and user preferences, all driven by outbox events.

push_devices

Column	Type	Rules / notes
<code>user_id / session_id</code>	CHAR(36) FK / CHAR(36) FK	Deleted when the session is revoked
<code>token</code>	VARCHAR(255) UNIQUE	FCM registration token; re-assigned if another user signs in on the device
<code>platform / app_version</code>	ENUM(android, ios) / VARCHAR(20)	
<code>last_seen_at</code>	DATETIME(3)	Tokens unseen 60 days are deleted

Indexes: (`user_id`)

notifications — in-app inbox

Column	Type	Rules / notes
<code>user_id</code>	CHAR(36) FK	
<code>type</code>	VARCHAR(40)	<code>match.new</code> , <code>like.received</code> , <code>security.new_device</code> , <code>verification.*</code> , <code>moderation.*</code> , <code>report.resolved</code> , <code>subscription.*</code>
<code>payload</code>	JSON	Ids only. The app fetches names through normal endpoints.
<code>dedupe_key</code>	VARCHAR(150) UNIQUE	e.g. <code>match.created:<matchId>:<userId></code>
<code>read_at</code>	DATETIME(3) NULL	

Indexes: (`user_id`, `created_at`)

notification_preferences — one per user, PK `user_id`

Column	Type	Rules / notes
<code>user_id</code>	CHAR(36) PK	
<code>matches / messages / likes</code>	BOOLEAN	Default true
<code>show_message_previews</code>	BOOLEAN	Default false : push says "New message from Priya", no text
<code>marketing</code>	BOOLEAN	Default false
<code>updated_at</code>	DATETIME(3)	

	Endpoint	Access	What it does
GET	<code>/notifications</code>	User	Inbox (cursor)
POST	<code>/notifications/read</code>	User	Body <code>{upToId}</code> or <code>{ids[]}</code>
GET	<code>/notifications/unread-count</code>	User	Badge number
PUT	<code>/notifications/devices</code>	User	Body <code>{token, platform, appVersion}</code> . Upsert, bound to this session.

Endpoint	Access	What it does
DELETE /notifications/devices/current	User	Remove this device's token
GET /notifications/preferences	User	
PATCH /notifications/preferences	User	Security and moderation categories cannot be switched off

- Provider: Firebase Cloud Messaging (Android and iOS via APNs key) through `firebase-admin`, behind a `PushProvider` adapter.
- Handler re-checks that the recipient can still authenticate; invalid-token responses delete the device row; message pushes use one collapse key per match.
- Inbox retention 90 days.

Done when: duplicate events never create duplicate notifications • pushes stop immediately after logout

Phase 6 • Monetization & trust+

After this phase plans can be bought, paid features unlock, contacts can be exchanged with consent, and users can earn the ID badge.

M21 Billing & Entitlements

Phase 6 (stub from Phase 4)

Depends on: M04, M03 **Tables:** 5 **APIs:** 7

Purpose. Sells plans through Google Play and the App Store, verifies every purchase with the store, and answers one question for the whole backend: what is this user allowed to do?

Plan matrix (entitlements, stored in `plans.entitlements`)

Entitlement key	Free	Basic ₹199	Plus ₹499	Premium ₹999
<code>daily_likes</code>	25	100	unlimited*	unlimited*
<code>see_who_liked_me</code>	count only	count only	full list	full list
<code>undo_pass</code> (per day)	0	3	unlimited	unlimited
<code>advanced_filters</code>	no	basic	all	all
<code>priority_likes</code>	0	1 / week	3 / week	1 / day
<code>boosts_per_month</code>	0	0	1	4
<code>contact_exchange_per_month</code>	0	2	10	unlimited*
<code>voice_video_calls</code>	no	no	yes	yes
<code>travel_mode</code>	no	no	yes	yes
<code>incognito</code>	no	no	no	yes

* unlimited means a fair-use ceiling from config (for example 500 likes / day). Always free on every plan: live verification, ID badge, chat in matches, block, report, unmatch, safety centre. Prices are monthly and include GST; quarterly and half-yearly products are extra rows in `plan_products`.

plans — PK code

Column	Type	Rules / notes
<code>code</code>	VARCHAR(20) PK	free, basic, plus, premium
<code>name / sort_order / is_active</code>	VARCHAR(50) / TINYINT / BOOLEAN	
<code>entitlements</code>	JSON	The matrix above, validated by a schema

plan_products — store products per plan and period

Column	Type	Rules / notes
<code>plan_code</code>	VARCHAR(20) FK plans	
<code>period</code>	ENUM(month, quarter, half_year)	
<code>price_paise</code>	INT UNSIGNED	Display price; the store price is authoritative
<code>android_product_id / ios_product_id</code>	VARCHAR(100)	Store SKUs
<code>is_active</code>	BOOLEAN	

subscriptions

Column	Type	Rules / notes
<code>user_id</code>	CHAR(36) FK	
<code>plan_code / product_id</code>	VARCHAR(20) / CHAR(36)	
<code>store</code>	ENUM(google_play, app_store)	

Column	Type	Rules / notes
<code>store_original_id</code>	VARCHAR(200) UNIQUE	Purchase token / original transaction id. One store purchase can link to only one user.
<code>status</code>	ENUM(active, grace_period, on_hold, cancelled, expired, refunded)	Cancelled = active until period end
<code>current_period_end</code>	DATETIME(3)	
<code>auto_renew</code>	BOOLEAN	

Indexes: (user_id, status), (status, current_period_end)

purchase_transactions — immutable ledger

Column	Type	Rules / notes
<code>user_id / subscription_id</code>	CHAR(36) / CHAR(36) NULL	
<code>store / store_transaction_id</code>	ENUM / VARCHAR(200) UNIQUE	Idempotency for webhooks
<code>type</code>	ENUM(purchase, renewal, refund, revoke)	
<code>amount_paise / currency</code>	INT / CHAR(3)	As reported by the store
<code>raw</code>	JSON	Minimal store payload for audit

boosts

Column	Type	Rules / notes
<code>user_id</code>	CHAR(36) FK	
<code>source</code>	ENUM(plan)	Granted monthly from the plan
<code>status</code>	ENUM(available, active, used, expired)	
<code>activated_at / expires_at</code>	DATETIME(3) NULL	Active for 30 min

Indexes: (user_id, status), (status, expires_at)

	Endpoint	Access	What it does
GET	<code>/billing/plans</code>	User	Plans, products, entitlements (for the paywall)
GET	<code>/billing/me</code>	User	Current plan, status, renewal date, entitlements, remaining quotas
POST	<code>/billing/purchases/verify</code>	User	Body {store, productId, purchaseToken} + Idempotency-Key. Verifies with the store, activates, acknowledges.
POST	<code>/billing/restore</code>	User	Re-links the user's store purchases after reinstall
POST	<code>/webhooks/google-play</code>	Hook	Real-time developer notifications (Pub/Sub push, verified token)
POST	<code>/webhooks/app-store</code>	Hook	App Store Server Notifications v2 (signed JWS verified)
POST	<code>/boosts/activate</code>	Verified	Starts an available boost

Rules

- `EntitlementsService.get(userId)` merges the Free plan with active subscriptions, cached in Redis for 5 min and invalidated on any subscription change. Code calls `has(key) / limit(key)`, never checks plan names.
- Never grant from app data alone. Google Play purchases must be acknowledged within 3 days or Google refunds them automatically.
- Webhooks drive renewals, grace periods, cancellations and refunds. An hourly job re-checks subscriptions near `current_period_end` in case a webhook was missed.
- Downgrade: stored paid filters are ignored, quotas drop at the next daily reset, already exchanged contacts stay exchanged.
- Account deletion does not cancel a store subscription; the API response and app tell the user to cancel it in the store.

Done when: fake purchase token is rejected • the same webhook twice changes nothing • refund removes entitlements within minutes

M22 Contact Exchange

Phase 6

Depends on: M16, M19, M21 **Tables:** 1 **APIs:** 4

Purpose. The only way phone numbers are shared: a paid user sends a request inside a match, and the other person decides.

contact_exchanges

Column	Type	Rules / notes
<code>match_id</code>	CHAR(36) FK matches	At most one pending request per match

Column	Type	Rules / notes
requester_user_id / recipient_user_id	CHAR(36) FK / CHAR(36) FK	
status	ENUM(pending, accepted, declined, cancelled, expired)	
responded_at / expires_at	DATETIME(3) NULL / DATETIME(3)	Pending requests expire after 72 h

Indexes: (match_id, status), (requester_user_id, created_at)

Endpoint	Access	What it does
POST /matches/:matchId/contact-exchange	Verified	Send a request. Needs the <code>contact_exchange_per_month</code> entitlement and quota.
POST /contact-exchanges/:id/accept	Verified	Recipient accepts (any plan, including Free). Both verified phone numbers are revealed.
POST /contact-exchanges/:id/decline	Verified	Recipient declines. Requester sees "not now".
GET /matches/:matchId/contact-exchange	Verified	Current state; phone numbers only if accepted and the match is still active

Rules

- Preconditions: match active and at least 24 h old, both users face-verified, each side has sent at least 3 messages (config), requester not discovery-restricted. Otherwise 422 `CONTACT_EXCHANGE_NOT_ALLOWED` with the missing condition.
- Only the **verified** phone from `users.phone` is shared, never a typed number. Shown as a `contact_card` system message visible only to the two users.
- After a decline the requester must wait 7 days before asking again in that match. Unmatch or block hides the card in the app (the number itself cannot be un-shared, which the accept screen explains).
- Quota counts requests sent in the calendar month. Accepting never costs the recipient anything.
- Once accepted, the chat contact-detail block (M19) is lifted for that match.

Done when: free user can accept but cannot request • numbers never returned after unmatch or block

M23 ID Verification (badge)

Phase 6

Depends on: M11, M09 **Tables:** 1 **APIs:** 3

Purpose. Optional government-ID check through a licensed KYC provider (DigiLocker-based). Earns the "ID verified" badge and confirms the real date of birth.

id_verifications

Column	Type	Rules / notes
user_id	CHAR(36) FK	
provider / provider_ref	VARCHAR(30) / VARCHAR(100)	Reference only
document_type	ENUM(aadhaar, pan, passport, driving_licence, voter_id)	
status	ENUM(created, pending, verified, failed, expired)	
name_matches	BOOLEAN NULL	First name on the document vs display name
face_matches	BOOLEAN NULL	Document photo vs verified selfie (provider-side)
verified_dob	DATE NULL	Only the date of birth is kept
consent_version / consented_at	VARCHAR(20) / DATETIME(3)	
decided_at	DATETIME(3) NULL	

Indexes: (user_id, created_at)

Endpoint	Access	What it does
POST /verification/id/start	Verified	Body { <code>documentType</code> , <code>consentVersion</code> }. Returns provider redirect URL or SDK token.
GET /verification/id	Verified	Status of the latest ID check
POST /webhooks/kyc	Hook	Provider result, signature verified; final status is also fetched server-side

Rules

- **Never store** document numbers, document images or full addresses. Only status, DOB, match flags and the provider reference.

- Verified: set `profiles.id_verified_at, dob_source = id_verified`, and replace the DOB. If the document shows under 18: ban with reason `underage` and raise a P0 case.
- Face mismatch with the selfie: status failed, flagged to moderation (possible stolen ID).
- Limit: 3 attempts per 30 days. The check is free for users (platform cost).

Done when: no ID number appears anywhere in DB, logs or S3 • underage document leads to ban and P0 case

Phase 7 • Launch hardening, then Phase 8 • Post-launch

Phase 7 is not a module: security review, load tests against section 6.3, data-deletion test, backup restore drill, and bug fixing. Phase 8 adds calls.

M24 Calls (voice & video)

Phase 8 (v1.1)

Depends on: M16, M19, M21 **Tables:** 1 **APIs:** 3

Purpose. In-app voice and video calls between matches without sharing phone numbers, through a calls provider (Agora or 100ms).

calls

Column	Type	Rules / notes
<code>match_id</code>	CHAR(36) FK	
<code>caller_user_id / callee_user_id</code>	CHAR(36) FK / CHAR(36) FK	
<code>type</code>	ENUM(voice, video)	
<code>status</code>	ENUM(ringing, ongoing, ended, missed, declined, failed)	
<code>provider_room_id</code>	VARCHAR(100)	
<code>answered_at / ended_at</code>	DATETIME(3) NULL	
<code>duration_sec</code>	INT UNSIGNED NULL	

Indexes: (`match_id, created_at`), (`callee_user_id, status`)

	Endpoint	Access	What it does
POST	<code>/matches/:matchId/calls</code>	Verified	Body <code>{type}</code> . Caller needs <code>voice_video_calls</code> . Returns room + short-lived token; rings callee via socket and push.
PATCH	<code>/calls/:callId</code>	Verified	Body <code>{action: answer decline end}</code> . Answer returns the callee's token.
GET	<code>/matches/:matchId/calls</code>	Verified	Call history for the match

- Callee can be on any plan. Both users must have sent at least one message; each user can turn off calls in settings.
- Tokens valid 2 minutes to join; ringing times out after 45 s (missed); max call length 60 min; no recording.

Done when: blocked or unmatched users cannot call • ended match terminates an ongoing call

9 Appendices

A. Domain events (outbox)

Event	Published by	Handled by
<code>user.registered</code>	M06	Analytics, welcome notification
<code>auth.new_device</code> • <code>auth.token_reuse</code>	M06	M20 security push + email
<code>user.status_changed</code>	M04 / M15	Revoke sessions, disconnect sockets, close matches on ban
<code>verification.face.approved</code> / <code>.rejected</code>	M11 / M15	M20 notification, onboarding next step
<code>verification.id.verified</code> / <code>.failed</code>	M23	M20 notification, moderation flag on mismatch
<code>photo.approved</code> / <code>.rejected</code> • <code>media.delete</code>	M12 / M15	M20 notification • S3 purge
<code>like.created</code>	M17	M20 aggregated like push
<code>match.created</code> • <code>match.ended</code>	M16	M20 push, realtime <code>match:new</code> / <code>match:ended</code>
<code>message.created</code> • <code>message.deleted</code>	M19	Realtime delivery, M20 push
<code>block.created</code>	M13	Cache invalidation
<code>report.created</code>	M14	Auto-restrict check, P0 paging
<code>moderation.action_taken</code>	M15	M20 notices to target and reporter
<code>subscription.activated</code> / <code>.changed</code> / <code>.expired</code>	M21	Entitlement cache invalidation, M20 notice
<code>contact_exchange.requested</code> / <code>.accepted</code>	M22	Realtime <code>contact:updated</code> , push
<code>call.started</code> • <code>call.ended</code>	M24	Realtime <code>call:incoming</code> , VoIP push
<code>account.deleted</code>	M07	Confirmation email (captured before scrub), purges

B. Background jobs

Job	Schedule	Module	What it does
Outbox relay	every 1 s	M03	Moves pending events into queues
Outbox cleanup	daily	M03	Deletes done events older than 7 days
OTP cleanup	hourly	M06	Deletes codes older than 24 h
Session cleanup	daily	M06	Deletes sessions expired or revoked > 30 days
Security events retention	daily	M05	Deletes events past retention
Suspension lift	every 5 min	M15	Re-activates users whose <code>suspended_until</code> passed
Photo / selfie purge	every 15 min	M12 / M11	Removes S3 objects of deleted items, retries failures
Selfie retention	daily	M11	Deletes rejected selfies after 30 days
Match reconciliation	every 10 min	M17	Creates matches missed for mutual likes (last 48 h)
Ended-match message purge	daily	M19	Hard-deletes messages 90 days after match end (unless under report)
Notification retention	daily	M20	Deletes inbox items older than 90 days
Push token cleanup	daily	M20	Deletes tokens unseen for 60 days
Subscription sweep	hourly	M21	Re-checks subscriptions near period end with the store
Monthly boost grant	daily	M21	Grants plan boosts on each renewal date
Boost expiry	every 5 min	M21	Ends boosts past <code>expires_at</code>
Contact request expiry	hourly	M22	Expires pending requests after 72 h
Data export builder	queue	M07	Builds ZIP, uploads, emails link

C. Error codes

Code	HTTP	When
VALIDATION_ERROR	400	Body or query failed validation (<code>details.errors[]</code>)
UNAUTHORIZED	401	Missing, invalid or revoked token
OTP_INVALID	401	Any OTP verify failure (same for every reason)
FORBIDDEN	403	Role not allowed
ACCOUNT_RESTRICTED	403	Suspended, banned or deactivated account
REAUTH_REQUIRED	403	Step-up OTP missing or older than 10 min
ONBOARDING_INCOMPLETE	403	Selfie not approved yet (<code>details.nextStep</code>)
ENTITLEMENT_REQUIRED	403	Feature not in the user's plan (<code>details.entitlement</code>)
USER_NOT_FOUND / PROFILE_NOT_FOUND / MATCH_NOT_FOUND	404	Not found, hidden, blocked or deleted (always the same)

Code	HTTP	When
REPORT_TARGET_INVALID	404	Report target or context not visible to reporter
PROFILE_ALREADY_EXISTS	409	Profile create when one exists
PHOTO_LIMIT_REACHED	409	More than 6 photos
DISCOVERY_NOT_READY	409	Viewer setup incomplete (<code>details.missing</code>)
INTERACTION_ALREADY_LIKED	409	Pass after like
CONTACT_EXCHANGE_PENDING	409	A request is already open for this match
PROFILE_DOB_LOCKED / UNDERAGE	422	DOB change attempt / age below 18
CONTACT_DETAILS_NOT_ALLOWED	422	Phone, email, UPI or handle in profile text
PHOTO_FACE_MISMATCH	422	Primary photo does not match verified face
MESSAGE_CONTENT_REJECTED	422	Empty or disallowed message
CONTACT_SHARING_LOCKED	422	Contact details in chat without an accepted exchange
CONTACT_EXCHANGE_NOT_ALLOWED	422	Preconditions not met (<code>details.reason</code>)
PURCHASE_INVALID / PURCHASE_ALREADY_LINKED	422	Store rejected the token / token belongs to another user
CALL_NOT_ALLOWED	422	Calls disabled or preconditions not met
PHOTO_INVALID_FILE	400	Not JPEG/PNG/WebP, too small or too large
OTP_COOLDOWN / TOO_MANY_REQUESTS	429	Rate limit (<code>details.retryAfterSeconds</code>)
LIKE_LIMIT_REACHED / MESSAGE_AWAITING_REPLY / VERIFICATION_ATTEMPTS_EXCEEDED	429	Quota reached
OTP_DELIVERY_FAILED / PROVIDER_UNAVAILABLE	503	SMS, email, face, KYC or store provider failed
INTERNAL_ERROR	500	Unexpected error (logged with requestId, no details to client)

D. Environment variables

Group	Variables
App	<code>NODE_ENV</code> , <code>PORT</code> , <code>APP_ROLE</code> (api realtime worker), <code>TRUST_PROXY</code> , <code>CORS_ORIGINS</code> , <code>LOG_LEVEL</code> , <code>SWAGGER_ENABLED</code>
Database	<code>DB_HOST</code> , <code>DB_PORT</code> , <code>DB_NAME</code> , <code>DB_USER</code> , <code>DB_PASSWORD</code> , <code>DB_POOL_MAX</code> , <code>DB_REPLICA_HOST</code> (optional)
Redis	<code>REDIS_URL</code> , <code>REDIS_TLS</code>
Auth	<code>JWT_ACCESS_SECRET</code> , <code>JWT_ACCESS_TTL</code> , <code>JWT_REFRESH_SECRET</code> , <code>SESSION_SLIDING_DAYS</code> , <code>SESSION_MAX_DAYS</code> , <code>OTP_HASH_SECRET</code> , <code>OTP_LENGTH</code> , <code>OTP_TTL_SECONDS</code> , <code>OTP_MAX_ATTEMPTS</code> , <code>OTP_RESEND_COOLDOWN_SECONDS</code> , <code>OTP_MAX_PER_HOUR</code> , <code>OTP_MAX_PER_IP_PER_HOUR</code> , <code>OTP_SMS_ALLOWED_COUNTRIES</code> , <code>OTP_DEV_ECHO</code> (refused in production)
SMS / Email	<code>SMS_PROVIDER</code> , <code>SMS_API_KEY</code> , <code>SMS_SENDER_ID</code> , <code>SMS_DLT_TEMPLATE_ID</code> , <code>SMS_DAILY_BUDGET</code> , <code>EMAIL_FROM</code> , <code>SES_REGION</code>
AWS / media	<code>AWS_REGION</code> , <code>S3_BUCKET_MEDIA</code> , <code>S3_BUCKET_PRIVATE</code> , <code>CLOUDFRONT_DOMAIN</code> , <code>CLOUDFRONT_KEY_PAIR_ID</code> , <code>CLOUDFRONT_PRIVATE_KEY</code> , <code>PHOTO_URL_TTL_SECONDS</code> (credentials via IAM role, never keys in production)
Verification	<code>FACE_PROVIDER</code> , <code>FACE_AUTO_APPROVE_SCORE</code> , <code>FACE_REVIEW_SCORE</code> , <code>FACE_MATCH_THRESHOLD</code> , <code>KYC_PROVIDER</code> , <code>KYC_API_KEY</code> , <code>KYC_WEBHOOK_SECRET</code>
Push	<code>FCM_PROJECT_ID</code> , <code>FCM_CLIENT_EMAIL</code> , <code>FCM_PRIVATE_KEY</code>
Billing	<code>GOOGLE_PLAY_PACKAGE_NAME</code> , <code>GOOGLE_PLAY_SERVICE_ACCOUNT_JSON</code> , <code>GOOGLE_RTDN_AUDIENCE</code> , <code>APPLE_BUNDLE_ID</code> , <code>APPLE_ISSUER_ID</code> , <code>APPLE_KEY_ID</code> , <code>APPLE_PRIVATE_KEY</code> , <code>APPLE_ENVIRONMENT</code>
Calls	<code>CALLS_PROVIDER</code> , <code>CALLS_APP_ID</code> , <code>CALLS_APP_CERTIFICATE</code>
Monitoring	<code>SENTRY_DSN</code> , <code>ALERT_WEBHOOK_URL</code>

Product limits (likes per plan, photo minimum, thresholds, windows) are **not** env variables. They live in `app_settings` so admins can change them at runtime.

E. Testing and Definition of Done

Test type	What is required
Unit	Every service rule and edge case. Target $\geq 80\%$ line coverage on <code>services/</code> .
Integration	Real MySQL and Redis via Testcontainers. Every query with a unique key, lock or race (OTP verify, refresh, like $\rightarrow$ match, message seq, purchases).
API / e2e	Each endpoint: success, validation error, not signed in, other user's resource (must be 404), and rate limit.
Security	IDOR check on every id in a path or body; blocked-user visibility; no PII in logs (log scan test).
Contract	OpenAPI file generated in CI and published for the app and QA teams; breaking changes fail the build.
Load	k6 scripts for discovery, like, send message and realtime delivery against the budgets in 6.3 before launch.

Definition of Done for every module

- All APIs in the module section implemented with Swagger docs, and every rule in its **Rules** list enforced by code and covered by a test.
- Migrations run cleanly on an empty DB and on a copy of staging; indexes checked with `EXPLAIN`.

- Account deletion handler registered (if the module stores user data) and covered by the deletion test.
- Security events recorded for every security-relevant action; no PII in logs.
- Config keys added to `app_settings` seed or env validation, and to Appendix D.
- Postman collection updated for QA; changelog entry written; reviewed and merged with green CI.

Where to go next. The App Developer Guide uses the same module codes, so an app screen, its API and its test case can always be traced to one module here. The QA Testing Guide lists test cases per module code. The Deployment, Security & Operations Guide covers AWS and VPS setup, domains, secrets, backups, monitoring and compliance.